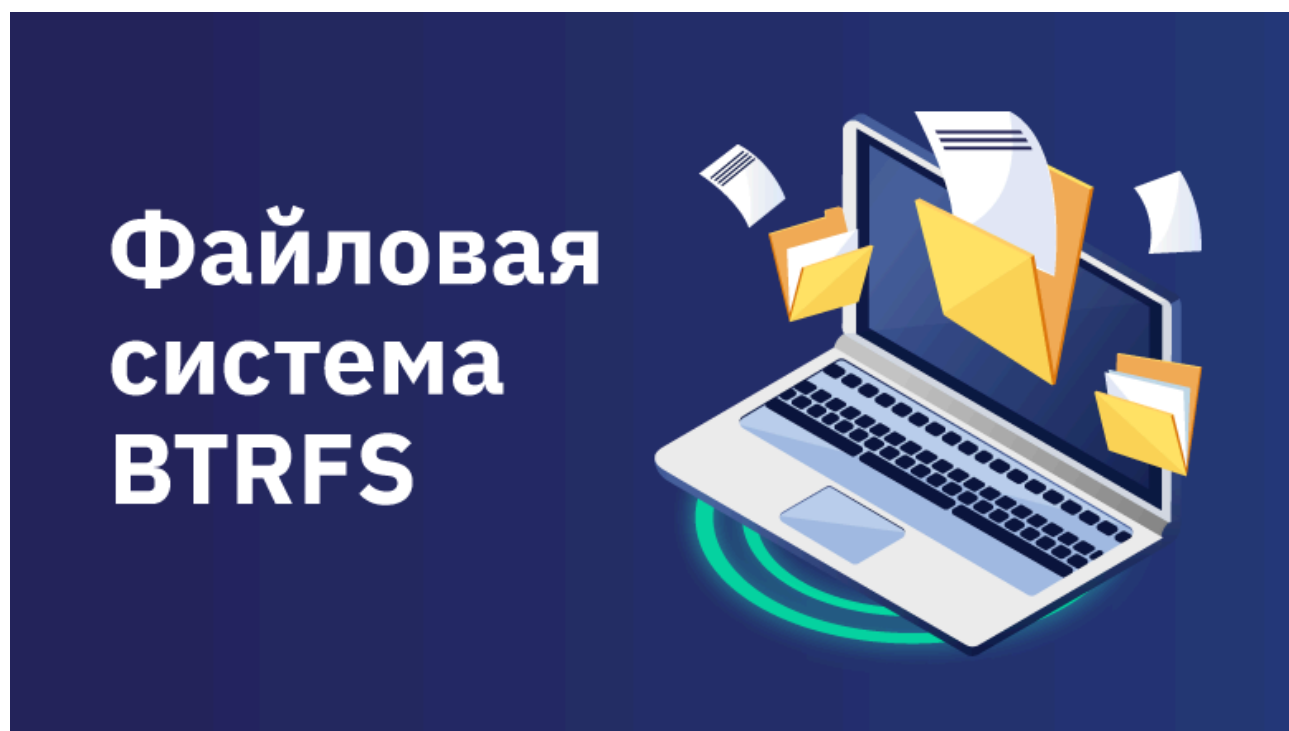


Файловая система BTRFS

 habr.com/ru/companies/otus/articles/725424

March 29, 2023



Средний

6 мин

57K

Обзор

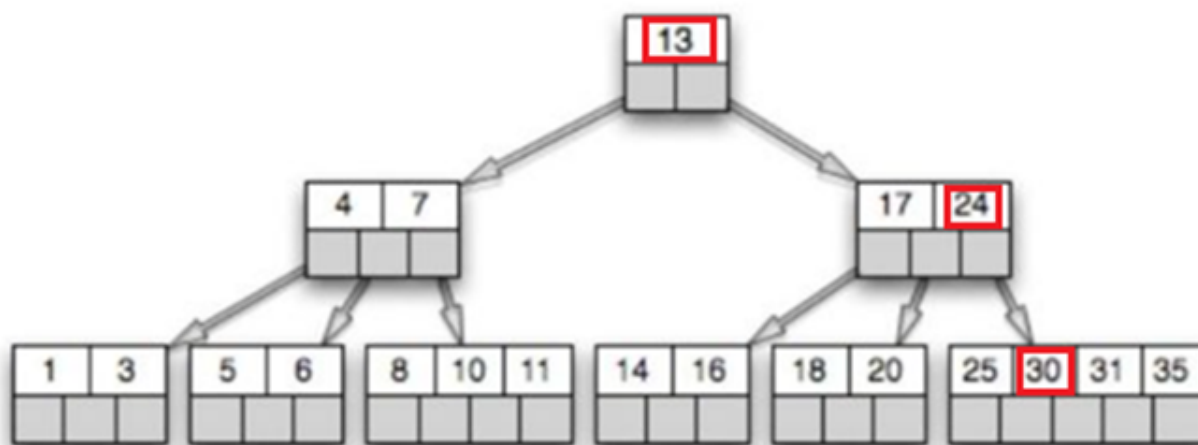
Некоторое время назад мной была представлена [статья, посвященная дисковой подсистеме ОС Linux](#) и среди прочих в комментариях к данной статье предлагалось рассмотреть работу с кэшем в файловой системе BTRFS. В этой статье я предлагаю вернуться к теме файловых систем в Linux и для начала посмотреть что из себя представляет BTRFS, где применяется и как с ней лучше работать. Данная статья предназначена для администраторов [Линукс](#), имеющих практический опыт администрирования данной ОС.

Итак, файловая система BTRFS (B-Tree Filesystem) предназначена для работы в Unix-подобных операционных системах. Она была разработана компанией Oracle в 2007 году. BTRFS построена по принципу CoW (Copy on Write), то есть при чтении области данных используется общая копия, в случае изменения данных — создается новая копия. Данная технология используется для оптимизации многих процессов, происходящих в операционной системе.

Немного о деревьях

В BTRFS как и в других ФС все данные хранятся на диске по определенным адресам, которые в свою очередь сохранены в метаданных. Однако, здесь все данные организуются в B-деревья (те самые B-tree, которые присутствуют в названии ФС). Не вдаваясь в дебри математических наук, которыми многих из нас мучали в вузе, кратко поясню принцип поиска в таких деревьях.

При поиске B-Tree мы должны выбирать пути из нескольких вариантов. То есть, если мы ищем в дереве например, значение 30, как представлено на картинке ниже, то при обходе мы сначала проверим ключ в корне, 13 меньше 30, значит нам надо перейти на правого потомка, $17 < 30$, $24 < 30$, значит снова переходим на правого потомка и здесь находим нужное значение.



Какое все это имеет отношение к работе файловой системы? B-Tree в отличие от двоичных деревьев позволяет хранить много ключей в одном узле и при этом может ссылаться на несколько дочерних узлов. Это значительно уменьшает высоту дерева и, соответственно, обеспечивает более быстрый доступ к диску. Но вернемся к BTRFS.

Основные характеристики

Прежде всего, как положено при описании какой-либо технологии или решения, приведу основные характеристики BTRFS. Максимальный размер файла в данной ФС может составлять 2^{64} байт или 16 экзбайт. При этом в BTRFS также реализована компактная упаковка небольших файлов и индексированные каталоги с экономией места. Для сжатия используются алгоритмы ZLIB, LZO, ZSTD, а также эвристика. Контрольные суммы для данных и метаданных вычисляются с помощью алгоритмов crc32c, xxhash, sha256, blake2b.

Ну а кроме этого, в BTRFS есть также динамическое распределение индексов, моментальные снимки с возможностью записи, снимки только для чтения и

вложенные тома (отдельные корни внутренней файловой системы). Далее мы более подробно поговорим об этих технологиях и начнем с работы со снимками.

Снимки в файловой системе

При работе с файлами, в частности, при операции перезаписи данных, они по факту не перезаписываются, а та часть данных, которая подверглась модификации копируется на новое место. Затем просто обновляются метаданные о расположении изменившейся информации. Благодаря такому решению мы можем создавать мгновенные снимки файловой системы, которые не занимают места на диске, пока не было внесено много изменений. Ну а если старый блок больше не нужен, потому что он не является частью какого-либо снимка, то мы можем его автоматически удалить.

Работа с дисками

Как мы уже упомянули ранее BTRFS может работать с файлами до 16 Экзабайт. Хотя текущая реализация VFS Linux (виртуальной файловой системы, то есть промежуточного слоя абстракции) имеет ограничение ядра до 8 Экзабайт. Но даже это уже огромный объем для работы с которым нужны соответствующие носители и BTRFS умеет эффективно работать с такими носителями. Большинство файловых систем используют диск целиком, от начала и до конца для записи своей структуры. Но BTRFS работает с дисками по-другому: независимо от размеров диска, мы делим его на блоки по 1 Гб для данных и 256 Мб для метаданных. Из этих блоков формируются группы, причем, каждая из которых может храниться на разных устройствах. При этом, количество таких блоков в группе может зависеть от уровня RAID для данной группы. Для управления блоками и группами используется специальное средство — менеджер томов, который уже интегрирован в файловую систему.

И еще одна небольшая плюшка. BTRFS при работе с небольшими файлами (по умолчанию до 8K) хранит их непосредственно в метаданных, тем самым снижая накладные расходы и уменьшая фрагментацию диска. Таким образом, наличие большого количества маленьких файлов, как минимум, не приведет к существенной просадке производительности.

Думаю, теории для этой статьи достаточно и сейчас самое время перейти к практике, а именно создать файловый раздел. В своем примере я буду по традиции использовать Ubuntu 22.04, однако другие современные дистрибутивы также должны поддерживать BTRFS

Разворачиваем BTRFS

Традиционно, процесс установки какого-либо ПО под Linux начинается с обновления пакетов:

```
sudo apt update
```

Далее в случае с Ubuntu 22.04 у меня поддержка ФС уже есть. Но в случае ее отсутствия необходимо было бы выполнить:

```
sudo apt install btrfs-progs -y
```

Далее нам потребуется неразмеченный диск. Посмотрим что у нас есть с помощью команды:

```
sudo lsblk -e7
```

В моем примере я буду работать с sdb размером 10 Гб.

```
root@otus:~# lsblk -e7
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                8:0    0   20G  0 disk
├─sda1                            8:1    0    1M  0 part
├─sda2                            8:2    0   1.8G  0 part /boot
└─sda3                            8:3    0  18.2G  0 part
   └─ubuntu--vg-ubuntu--lv 253:0    0    10G  0 lvm  /
sdb                                8:16    0   10G  0 disk
sr0                                11:0    1   1.4G  0 rom
```



Так как мы будем мучать sdb, то укажем его в параметрах утилиты cfdisk:

```
sudo cfdisk /dev/sdb
```

После запуска утилиты выбираем gpt.


```

root@otus:~# mkfs.btrfs -f -L btrfs /dev/sdb
btrfs-progs v5.16.2
See http://btrfs.wiki.kernel.org for more information.

NOTE: several default settings have changed in version 5.15, please make sure
this does not affect your deployments:
- DUP for metadata (-m dup)
- enabled no-holes (-O no-holes)
- enabled free-space-tree (-R free-space-tree)

Label:                btrfs
UUID:                 d4294167-19b4-4e6a-bbfe-4225e952c72c
Node size:            16384
Sector size:          4096
Filesystem size:      10.00GiB
Block group profiles:
  Data:               single             8.00MiB
  Metadata:           DUP                256.00MiB
  System:             DUP                8.00MiB
SSD detected:         no
Zoned device:         no
Incompat features:    extref, skinny-metadata, no-holes
Runtime features:     free-space-tree
Checksum:             crc32c
Number of devices:    1
Devices:
  ID     SIZE  PATH
  1     10.00GiB /dev/sdb

```

В результате мы получили полную информацию об отформатированном диске. Теперь подмонтируем BTRFS раздел. Для этого сначала создаем точку монтирования. В моем случае это каталог /btrfs. А затем подмонтируем sdb к этому разделу.

```
sudo mkdir -v /btrfs
```

```
sudo mount /dev/sdb /btrfs
```

И для того, чтобы убедиться в корректности выполненных операций, воспользуемся уже знакомой нам командой:

```
sudo lsblk -e7
```

Статистика использования

Поговорим немного о том, как можно смотреть различную статистику использования BTRFS дисков. Прежде всего воспользуемся командой:

```
sudo btrfs filesystem usage /data
```

```

root@otus:/btrfs# btrfs filesystem usage /btrfs
Overall:
  Device size:                10.00GiB
  Device allocated:           536.00MiB
  Device unallocated:         9.48GiB
  Device missing:             0.00B
  Used:                       356.00KiB
  Free (estimated):           9.48GiB      (min: 4.75GiB)
  Free (statfs, df):          9.48GiB
  Data ratio:                 1.00
  Metadata ratio:             2.00
  Global reserve:             3.25MiB      (used: 0.00B)
  Multiple profiles:          no

Data,single: Size:8.00MiB, Used:68.00KiB (0.83%)
  /dev/sdb          8.00MiB

Metadata,DUP: Size:256.00MiB, Used:128.00KiB (0.05%)
  /dev/sdb          512.00MiB

System,DUP: Size:8.00MiB, Used:16.00KiB (0.20%)
  /dev/sdb          16.00MiB

Unallocated:
  /dev/sdb          9.48GiB

```

Вывод команды содержит различные значения. Посмотрим что означает каждое из них. В первой строке у нас идет общий размер диска — 10 Гб. Далее мы видим объем дискового пространства, зарезервированного для хранения данных и еще неиспользуемого пространства, которого естественно намного больше, так как пока на диск записано немного данных — сколько именно мы видим из значения поля Used. Отрадно, что значение Device missing равно 0.

Далее указаны методы (single или DUP), которые используются для выделения дискового пространства для данных, метаданных и системных данных.

При использовании single ФС хранит только одну копию данных, никакой дубликации не используется. Но при использовании DUP файловая система Btrfs выделит для одних и тех же данных дисковое пространство в разных частях файловой системы. Таким образом, в файловой системе будет храниться несколько копий (обычно две) одних и тех же данных.

Как можно видеть из рисунка выше, обычные данные у нас распределены в single режиме, а вот метаданные и системные данные используют DUP.

Заключение

В этой статье мы начали рассмотрение файловой системы BTRFS. Поговорили об основных преимуществах этой ФС и рассмотрели создание раздела

под управлением BTRFS. В заключении хотелось бы заметить, что в сети можно встретить негативные отзывы об использовании данной ФС. Однако, все эти отзывы датированы началом 2010-х годов, когда система еще не была стабильна и могли происходить сбои. Но сейчас эта ФС поддерживается многими операционными системами, ее спецификация уже много лет не изменяется и ее можно использовать без опасения потерять свои данные.

В следующей статье мы продолжим рассмотрение BTRFS, и в частности поговорим об использовании нескольких дисков и создании отказоустойчивых конфигураций.



Работаете с Linux, но в продакшене уже нужны не базовые команды, а уверенное понимание файловых систем, дисков, отказоустойчивости и настройки системы под реальные задачи? [Курс «Администратор Linux. Продвинутый уровень»](#) помогает закрыть этот разрыв и перейти от простого администрирования к сильной системной практике.

Проверьте, готовы ли вы к продвинутым задачам в Linux и где у вас реальные пробелы.

➡ [\[Пройти тестирование\]](#).